

동적 안정성을 고려한 강화학습기반 7DOF 로봇 암의 경로 최적화

Python과 pybullet의 Franka Panda모델 활용

홍근영

Kiros 7기

목차



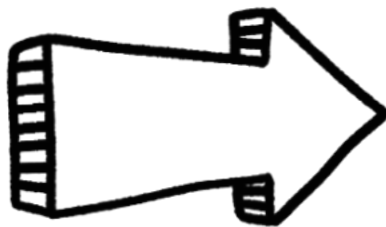
- 01 주제 선정 배경
- 02 프로젝트 목적 및 목표
- 03 프로젝트 개요
- 04 코드 및 관련 이론
- 05 결과 분석
- 06 결론 및 고찰

01 주제 선정 배경



1. 낮은 정확성

2. 동적 불안정성



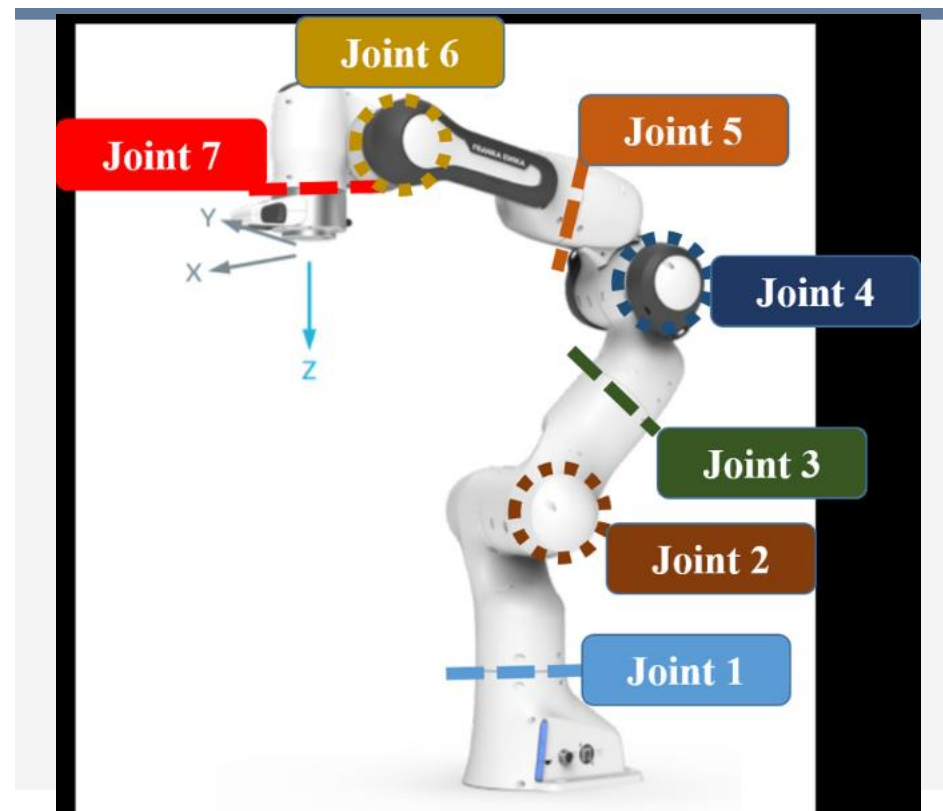
동작 안정성을 고려한 강화학습기반
7축 로봇암의 경로 최적화

02 프로젝트 목적 및 목표

목적 및 목표

목적 : 동적 안정성을 극복한 AI기반
정밀 파지 시스템 개발

목표 : 에너지 효율과 역학적 안정성
이 뛰어난 5mm이내의 파지 시스템
개발



03 프로젝트 개요

1. 환경 설정

2. 랜덤 위치에 목표 설정

3. 보상 함수 설정

4. 학습 및 테스트

04 코드 및 관련 이론

환경

```
import gymnasium as gym
from gymnasium import spaces
import pybullet as p
import pybullet_data as pd
import numpy as np
import time
import os
```

```
# Stable Baselines3
from stable_baselines3 import SAC
from stable_baselines3.common.vec_env import SubprocVecEnv, VecMonitor
from stable_baselines3.common.env_util import make_vec_env
```

강화학습 에이전트가

관측(Observation)→행동(Action)→보상(Reward)→다음
관측을 반복하는 MDP를 표준화된 API로 정의·실행할 수
있게 해주는 인터페이스

04 코드 및 관련 이론

DOF

-Uniqueness of solutions

→ Arm DOFs : $n = 6 \Rightarrow n = \text{number of equations (=6)}$

- End-effector can be placed at arbitrary poses in workspace
- Unique inverse kinematic solutions exist

→ Arm DOFs : $n > 6 \Rightarrow n > \text{number of equations (=6)}$

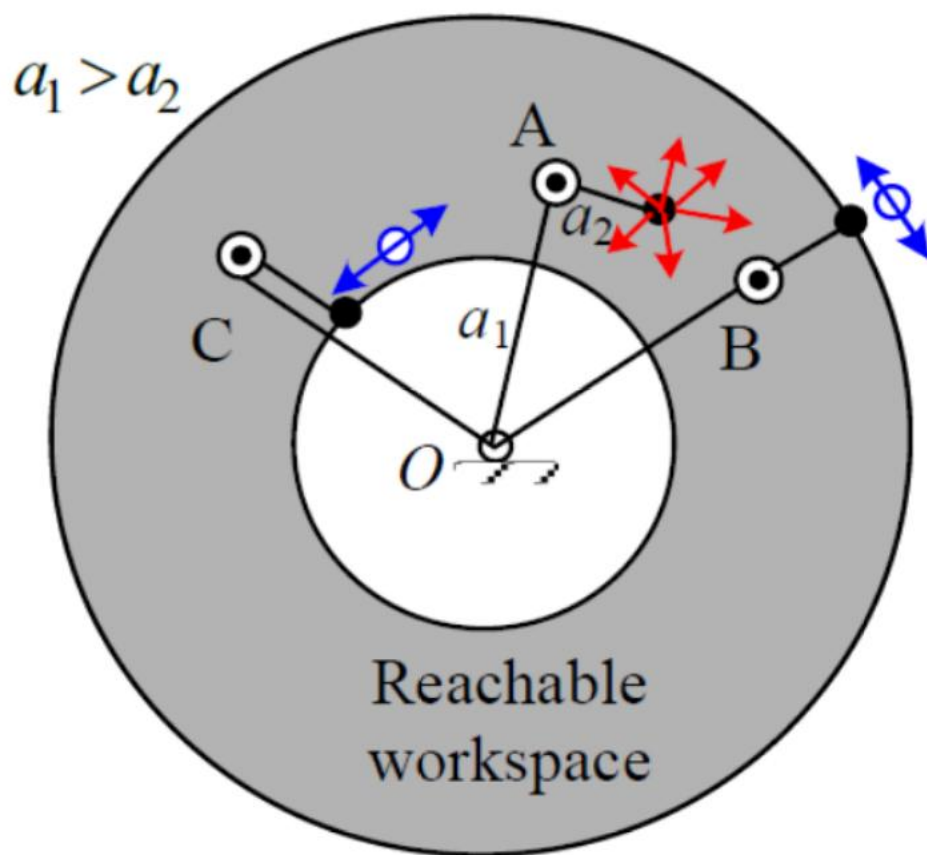
- End-effector can be placed at arbitrary poses in workspace
- Infinite(multiple) inverse kinematic solutions can exist $\Rightarrow$ kinematically redundant
- Redundancy provides flexibility with which robot can avoid obstacles

→ Arm DOFs : $n < 6 \Rightarrow n < \text{number of equations (=6)}$

- End-effector cannot be placed at certain poses in workspace

04 코드 및 관련 이론

Singularity



$$\rightarrow \begin{Bmatrix} v_x \\ v_y \end{Bmatrix} = \begin{bmatrix} -(a_1 s_1 + a_2 s_{12}) & -(a_2 s_{12}) \\ (a_1 c_1 + a_2 c_{12}) & (a_2 c_{12}) \end{bmatrix} \cdot \begin{Bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{Bmatrix}$$

$$\rightarrow \det(J) = a_1 a_2 \sin \theta_2 = 0 \Rightarrow \theta_2 = 0^\circ \text{ or } 180^\circ$$

$$\rightarrow \theta_2 = 0^\circ \Rightarrow \text{Fully extended (B, stretched)}$$

$$\rightarrow \theta_2 = 180^\circ \Rightarrow \text{Fully contracted (C, folded back)}$$

$\rightarrow$ End-effector velocities in a certain direction cannot be realized by any finite joint velocity

* Mathematical background (Nonsingular)

$$\rightarrow \text{rank}(\mathbf{A}) = n$$

$$\rightarrow \det(\mathbf{A}) \neq 0$$

$$\rightarrow \mathbf{A} : \text{invertible (i.e., } \mathbf{A}^{-1} \text{ exists)}$$

04 코드 및 관련 이론

가동 범위

```
# 2. 행동(Action)과 관측(Observation) 공간 정의
# 행동: 7개 관절의 속도/위치 변화량 (-1.0 ~ 1.0)
self.action_space = spaces.Box(low=-1.0, high=1.0, shape=(7,), dtype=np.float32)
# 관측: 20개 데이터 (관절각도 7 + 속도 7 + 손끝위치 3 + 목표상대위치 3)
self.observation_space = spaces.Box(low=-np.inf, high=np.inf, shape=(20,), dtype=np.float32)

# 3. 로봇 하드웨어 설정 (관절 인덱스 및 한계값)
self.joint_indices = [0, 1, 2, 3, 4, 5, 6]
self.ee_link_index = 11 # Panda 로봇의 손끝(End-Effector) 링크 번호

# 관절 각도 하한(ll)과 상한(ul) - 로봇이 꺾일 수 있는 물리적 한계
self.ll = np.array([-2.89, -1.76, -2.89, -3.07, -2.89, -0.01, -2.89])
self.ul = np.array([ 2.89,  1.76,  2.89, -0.06,  2.89,  3.75,  2.89])
self.joint_range = self.ul - self.ll
```

04 코드 및 관련 이론

보상 체계 및 함수

```
def step(self, action):
    # 1. 행동 스케일링 (너무 급격하게 움직이지 않도록 0.03 곱함)
    real_action = action * 0.03

    # 2. 모터 제어 명령 (Position Control)
    # 현재 각도 + 행동(변화량) = 목표 각도
    p.setJointMotorControlArray(..., targetPositions=target_q, forces=[500]*7)

    # 3. 물리 엔진 업데이트 (0.05초 동안 시뮬레이션 진행)
    for _ in range(int(self.control_dt / self.sim_step)):
        p.stepSimulation()

    # 4. 관측 및 거리 계산
    obs = self._get_obs()
    distance = np.linalg.norm(self.target_pos - ee_pos)
```

```
# 5. [보상 설계]
# (1) 거리 보상: 가까울수록 점수가 덜 깎임 (음수 보상)
reward = -distance * 20.0

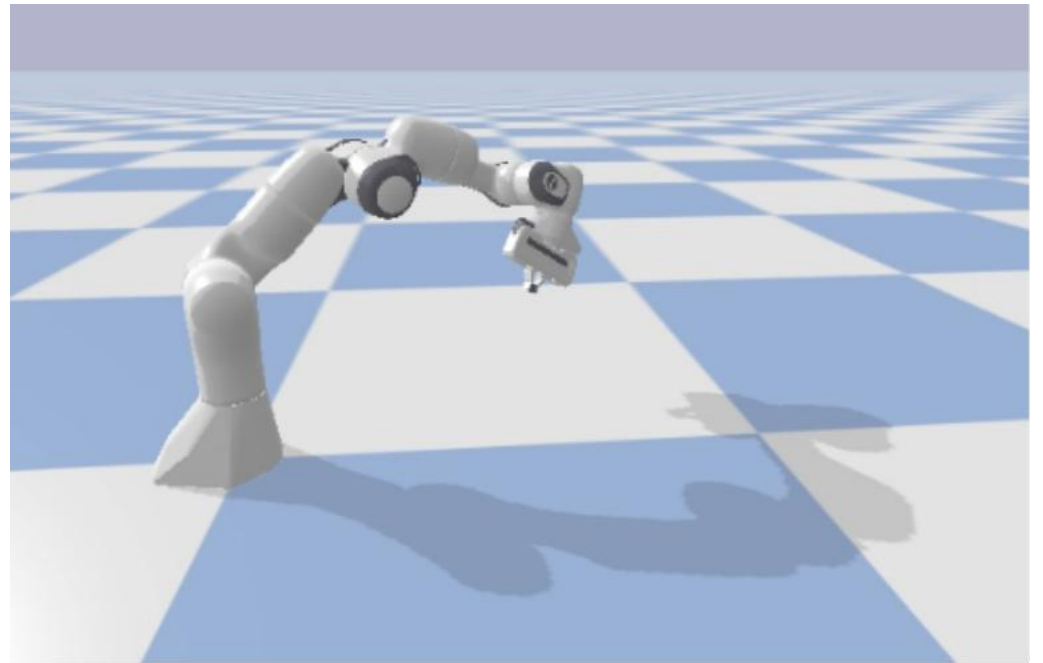
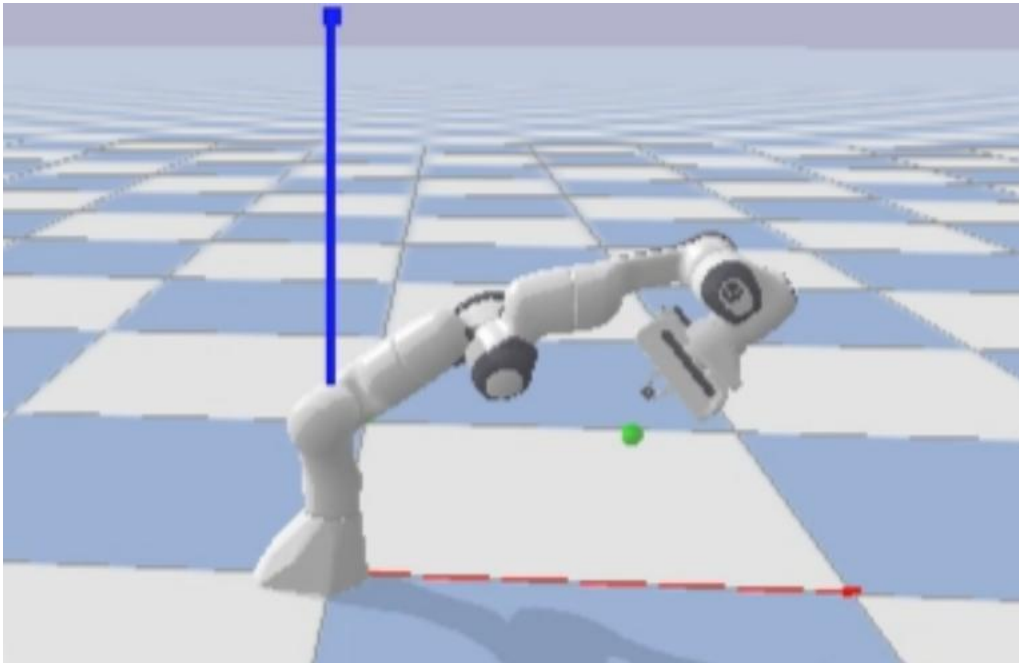
# (2) 정밀 접근 보상: 아주 가까워지면 추가 점수
if distance < 0.02:
    reward += (0.02 - distance) * 200.0

# (3) 에너지 패널티: 힘(Torque)을 많이 쓰거나, 급격히 움직이면 감점
reward -= 0.01 * np.mean(np.square(self.current_torques))
reward -= 0.1 * np.mean(np.square(action - self.prev_action))

# 6. 종료 조건 (성공 또는 충돌)
if len(contact_points) > 0: # 충돌 시
    reward -= 10.0
    terminated = True
elif distance < 0.005: # 목표 도달 성공 시
    terminated = True
    reward += 150.0 # 큰 성공 보상

return obs, reward, terminated, truncated, {"distance": distance}
```

05 결과 분석



06 결론 및 고찰

목표지점에 95% 확률로 5mm이내로 접근
또한, 나머지 4%는 약 1cm 이내로 접근하며 좋은 정확도를 가진다고 판단할 수 있습니다.

프로젝트를 진행하며 대략 총 3억번 정도 학습을 수행하였고 강화학습이 어떤 것 인지, 보상 함수 설계가 얼마나 중요한지 알 수 있게 되었습니다.

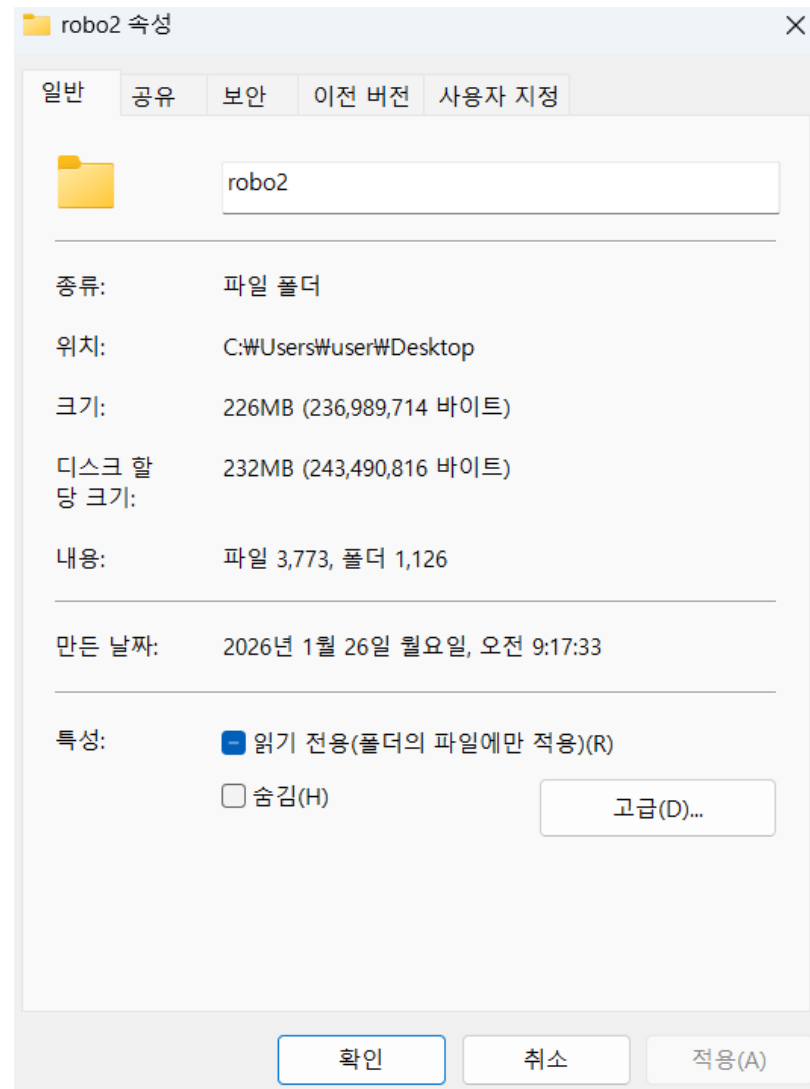
그러나 좋은 정확도를 가진 모델을 학습시켰지만, 이 모델이 역학적으로 안정적인 자세로 목표점에 도달했는가에 대한 의문은 해소하지 못했습니다.

그래서 다음 개인 프로젝트 주제는 이번에 학습한 모델이 역학적으로 안정적인 모델인가에 대한 주제로 진행하려고 합니다.

Pro 모델 한도에 도달했습니다

1월 25일 오전 6:45에 한도가 리셋될 때까지 대답에 다른 모델이 사용됩니다. 한도를 늘리고 더 많은 기능을 이용하려면 업그레이드하세요.

업그레이드



감사합니다.

홍근영

Kiros 7기